

Learn Python for Automation

Lesson 4: Functions and Modules

Functions and Modules - Study Notes

Key Concepts

1. **Function definition** – `def` keyword, name, parameters, and body.
2. **Arguments vs. parameters** – the values passed in (`arguments`) versus the placeholders defined in the function signature (`parameters`).
3. **Return values** – `return` statement sends a value back to the caller; if omitted, `None` is returned.
4. **Scope** – local variables inside a function vs. global variables outside.
5. **Importing modules** – `import` statement brings in standard library modules (e.g., `math`, `random`) so you can reuse pre-written code.

Summary

Functions are reusable blocks of code that take input, perform a task, and optionally return a result. They are defined using the `def` keyword, followed by a name, a list of parameters in parentheses, and a colon. The indented block that follows is the function body. Parameters act as placeholders for the arguments supplied when the function is called. Inside the body, you can perform calculations, call other functions, and use `return` to send data back to the caller. If no `return` is specified, the function implicitly returns `None`.

Modules are packages of Python code that group related functions, classes, and variables. The Python Standard Library offers many useful modules (e.g., `math` for mathematical functions, `datetime` for date/time operations). Importing a module with `import module_name` makes its contents available in your script. You can also import specific objects with `from module_name import object_name` or give a module an alias using `as`. Using modules reduces code duplication and leverages battle-tested implementations.

Important Points

- **Defining a function**

```
python
def greet(name):
    print(f"Hello, {name}!")
python
```

- **Calling a function**

```
python
greet("Alice") # prints: Hello, Alice!
python
```

- **Parameters vs. arguments**

- **Parameters**: `name` in the definition.

- ***Arguments***: `"Alice"` in the call.

- ****Return values****

```
```python
def add(a, b):
 return a + b
result = add(3, 4) # result == 7
```
```

- ****Scope rules****

- Variables defined inside a function are local.

- Use ``global`` keyword to modify a global variable inside a function (rarely needed).

- ****Importing modules****

```
```python
import math
print(math.sqrt(16)) # 4.0
```
```

- ****Aliasing modules****

```
```python
import numpy as np
array = np.array([1, 2, 3])
```
```

- ****Import specific names****

```
```python
from random import randint
print(randint(1, 10))
```
```

- ****Avoiding namespace clashes****

Use ``as`` to give a module a unique alias, especially if two modules share a name.

Examples

1. Simple Calculator Function

```
```python
def calculator(op, x, y):
 """Perform basic arithmetic operations."""
 if op == '+':
 return x + y
 elif op == '-':
 return x - y
 elif op == '*':
 return x * y
 elif op == '/':
 return x / y
```
```

```

else:
    raise ValueError("Unsupported operation")
print(calculator('*', 6, 7)) # Output: 42
...

```

Explanation: The function takes an operator and two numbers, uses conditional logic to decide which operation to perform, and returns the result.

2. Using the `math` Module

```

```python
import math
def circle_area(radius):
 """Return the area of a circle given its radius."""
 return math.pi * radius ** 2
print(circle_area(5)) # Output: 78.53981633974483
...

```

\*Explanation:\* The `math` module provides the constant `pi`. The function calculates area using the formula  $A = \pi r^2$  where  $r$  is the numeric value.

---

## Quick Review

1. What keyword starts a function definition?
2. How do you call a function named `my\_func` with two arguments, 10 and 20?
3. What happens if a function reaches the end of its body without encountering a `return` statement?
4. Write the import statement to bring in the `datetime` module and give it the alias `dt`.
5. Explain the difference between `import math` and `from math import sqrt`.

---

## Further Reading

- **Advanced function concepts**: default arguments, keyword arguments, variable-length arguments (`\*args`, `\*\*kwargs`), lambda functions.
- **Scope and namespaces**: global vs. local, `nonlocal` keyword.
- **Module creation**: how to write your own modules and packages.
- **The `os` and `sys` modules**: interacting with the operating system and command line arguments.
- **Docstrings and the `help()` function**: documenting and introspecting functions.

